BÖLÜM 3 / MODULE 3

Klasik Senkronizasyon, Monitörler & Kilitler

Classical Synchronization, Monitors & Locks

synchronized iç kilit mekanizması (Intrinsic Lock), HotSpot monitör seviyeleri (Biased, Lightweight, Heavyweight), wait/notify signal tuzakları, ReentrantLock, Condition kuyrukları ve ReentrantReadWriteLock.

JVM intrinsic locks via synchronized, HotSpot lock inflation levels, wait/notify missed signals, ReentrantLock reentrancy, Condition queues, and ReadWriteLock degradation.

13

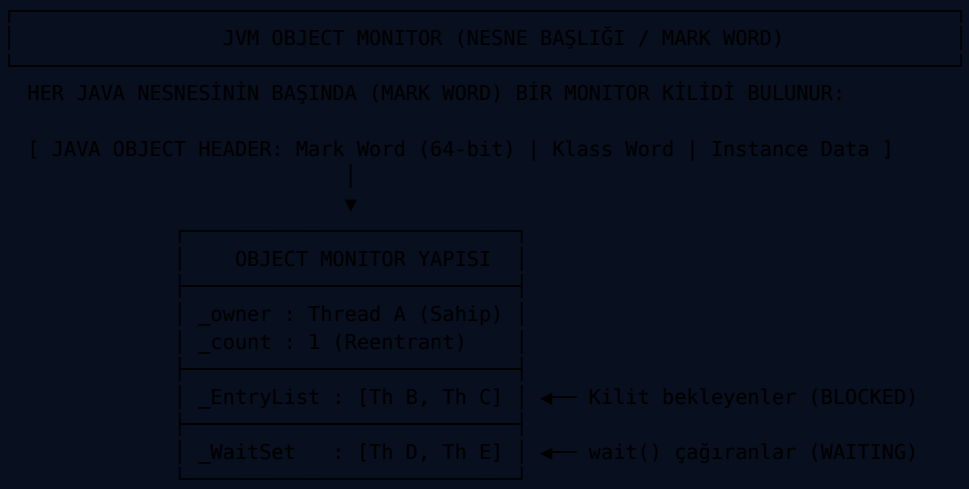
BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

synchronized Anahtar Kelimesi: Intrinsic Lock & JVM Object Monitor Mimarisi

The synchronized Keyword: Intrinsic Locks & JVM Object Monitors

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Her Java nesnesi, kapısında özel bir anahtar deliği olan bir oda gibidir. Bir odaya `synchronized` ile girdiğinizde (monitorenter), içeriden kapıyı kilitlersiniz. Odanın sahibi siz olursunuz (_owner). Kapıda iki tane bekleme salonu vardır: 1) Koridor (_EntryList): İçeri girmek için sıraya girenler (BLOCKED). 2) Balkon (_WaitSet): İçerideyken 'Patron haber verene kadar burada dinleniyorum' diyerek kenara çekilenler (WAITING). Siz odadan çıkarken (monitorexit) anahtarı bırakırsınız ve koridordaki bir kişiyi içeri alırsınız!

🇬🇧 ENGLISH (ELI5)

Every Java object is born with a private hotel room keyhole inside its memory header (Mark Word). When a thread enters a `synchronized` block, it executes `monitorenter`, turning the deadbolt from the inside and claiming ownership (`_owner`). Outside sit two distinct waiting rooms: 1) The Hallway (`_EntryList`): Threads aggressively competing to enter the room (BLOCKED). 2) The Balcony (`_WaitSet`): Threads that were inside, voluntarily surrendered the key via `wait()`, and are resting until signaled (WAITING). Exiting triggers `monitorexit`, unlocking the door for hallway contenders.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Object Monitor ve Bytecode Derinliği: 1) Mark Word: 64-bit HotSpot JVM'de nesne başlığının en alt 2-biti kilit durumunu tutar (01: Kilitsiz, 00: Hafif Kilit, 10: Ağır Monitör Kilidi). 2) Reentrancy (Yeniden Girilebilirlik): Bir thread zaten sahip olduğu bir kilide tekrar girerse (örneğin synchronized metodun içinden başka bir synchronized metot çağırmak), JVM `_count` sayacını 1 artırır; kilitlenip deadlock olmaz. Çıkarken de sayaç düşer. 3) Derleyici Garantisi: `synchronized` bloğu javac tarafından try-finally bloğu gibi derlenir. Metot içinde beklenmedik bir RuntimeException veya Error fırlasa dahi JVM `monitorexit` talimatını kesinlikle çalıştırır; kilit asla askıda kalmaz.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Monitor Internals & Bytecode: 1) Mark Word State Bits: On 64-bit architectures, the lowest two bits designate monitor states (01 = Unlocked, 00 = Light-weight/Thin, 10 = Heavyweight Inflated Monitor). 2) Reentrancy Mechanism: If Thread A holds a monitor and enters another synchronized block on the same object, the JVM simply increments `_count`. It does not re-acquire or block, preventing self-deadlock. The monitor is released when `_count` hits zero. 3) Bytecode Invariant: javac generates dual `monitorexit` opcodes: one for normal control flow completion and another wrapped in an all-encompassing exception handler, ensuring monitors are guaranteed released even under fatal panics.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Synchronized Reentrancy ve Monitor Davranışı
public class SynchronizedMonitorDemo {
    private final Object lock = new Object();
    private int sharedBalance = 1000;

    public void withdraw(int amount) {
        synchronized (lock) { // monitorenter
            IO.println("Kilit alındı: " + Thread.currentThread().getName());
            auditLog(); // Reentrant çağrı (Aynı kilit, sayaç 2 olur!)
            sharedBalance -= amount;
        } // monitorexit
    }

    private void auditLog() {
        synchronized (lock) { // Aynı thread olduğu için bloke olmaz!
            IO.println("Audit kaydı işlendi. Kalan bakiye: " + sharedBalance);
        }
    }

    void main() {
        var demo = new SynchronizedMonitorDemo();
        demo.withdraw(250);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'synchronized(new Object()) veya synchronized(new String("lock")) yazarsak ne olur?' Yanıt: 'Her thread farklı bir nesne referansı üzerinde kilitleneceği için hiçbir senkronizasyon sağlanamaz! Kilit nesneleri DAİMA `private final Object lock = new Object();` olarak tanımlanmalıdır.'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Junior Trap in Interviews: 'What is the catastrophe of `synchronized(new Object())` or `synchronized("literal")`?' Answer: '`new Object()` creates a brand-new monitor per invocation, providing ZERO mutual exclusion. Locking on interned Strings risks accidental cross-library deadlocks. Always declare `private final Object lock = new Object();`.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Her Java nesnesi başlığında (Mark Word) bir Object Monitor taşır.
- synchronized kilitleri reentrant'tır; aynı thread aynı kilide defalarca girebilir.
- Kilit nesneleri referans değişimini önlemek için daima private final olmalıdır.

🎯 Key Takeaways for the Interview

- Every Java object carries an intrinsic monitor inside its Mark Word header.
- Intrinsic locks are fully reentrant, incrementing an internal recursion counter.
- Lock target instances must always be private and final to prevent reference mutation.

14

BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

HotSpot Kilit Şişmesi: Biased Locking, Lightweight Locking & Heavyweight Monitor

HotSpot Lock Optimization: Biased, Lightweight (Thin) & Heavyweight Lock Inflation

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                HOTSPOT KİLİT ŞİŞMESİ (LOCK INFLATION) AŞAMALARI │
└──────────────────────────────────────────────────────────────┘

1. BIASED LOCKING (Taraflı Kilit - Tarihsel / Tek Thread):
   [ Nesne Başlığı ──► Thread ID yazılır ]
   Kilit maliyeti = 0 (Sadece tek bir thread giriyorsa sıfır senkronizasyon maliyeti).

2. LIGHTWEIGHT LOCK (Hafif Kilit / Çekişmesiz Çoklu Thread):
   Farklı thread'ler giriyor ama AYNI ANDA DEĞİL (Alternating execution).
   [ Nesne Başlığı ──► Thread'in kendi Stack'indeki BasicObjectLock'a işaret eder (CAS) ]
   İşletim sistemi kernel'ına İNİLMEZ, tamamen user-space CAS ile çözülür!

3. HEAVYWEIGHT MONITOR (Ağır Kilit / Çekişmeli Durum):
   İki thread AYNI ANDA kilide saldırır! CAS başarısız olur.
   [ Nesne Başlığı ──► OS seviyesinde ObjectMonitor C++ nesnesine işaret eder ]
   Thread'ler OS kernel seviyesinde BLOCKED durumuna alınır ve uyutulur (Park).
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirketteki toplantı odasını kiralama adımları gibidir: 1) Taraflı Kilit: Odayı hep aynı genel müdür kullanıyorsa, kapıya genel müdürün resmi asılır. Müdür kapıyı çalmadan doğrudan girer (Sıfır maliyet). 2) Hafif Kilit: Odayı ara sıra farklı çalışanlar kullanıyor ama hiç kimse birbiriyle çakışmıyor. Kapıya basit bir mandal konur (Kullanıcı seviyesinde CAS). 3) Ağır Kilit: İki yönetici aynı anda odaya girmeye çalışır ve kapıda kavga çıkar! Güvenlik (OS Kernel) çağrılır, yöneticilerden biri zorla bekleme salonuna oturtulup uyutulur (Context Switch maliyeti)!

🇬🇧 ENGLISH (ELI5)

HotSpot optimizes locking like an escalating conference room protocol: 1) Biased Locking: If Manager A is the sole user, their nameplate stays on the door; they walk in without checking locks (zero cost). 2) Lightweight (Thin) Locking: Different team members book the room, but their schedules never collide. They flip a simple physical door latch using atomic CAS in user space without security guards. 3) Heavyweight Monitor: Two directors arrive simultaneously and collide at the door threshold! Armed security (the OS kernel) intervenes, putting one director into forced sleep in the waiting lounge.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HotSpot Kilit Optimizasyon Kademeleri: 1) Kilit Şişmesi (Inflation) Tek Yönlüdür: HotSpot'ta kilit Hafif'ten Ağır'a şiştiğinde (Inflatable), genellikle JVM yeniden başlatılana kadar ağır kalır. 2) Lightweight Locking (CAS Lock): Thread, kendi thread stack'inde bir `BasicObjectLock` açar ve nesnenin Mark Word'ünü CAS ile bu stack kaydına yönlendirir. Çekişme yoksa tek bir CAS ile kilit alınır ve bırakılır. 3) Spin-Locking (Uyarlamalı Dönme / Adaptive Spinning): Ağır kilide geçmeden önce HotSpot thread'i hemen uyutmaz; kısa bir süre döngüde (spin) bekler. Eğer kilidi tutan diğer thread hemen işini bitirirse, pahalı işletim sistemi context-switch maliyetinden kurtulunur! 4) Biased Locking Durumu: Modern JDK sürümlerinde (JDK 15+) Biased Locking, safepoint maliyetleri nedeniyle varsayılan olarak kapatılmıştır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Lock Optimization Architecture: 1) One-Way Inflation: Once contention forces an object's Mark Word to point to an inflated C++ `ObjectMonitor`, deflation rarely occurs until safepoint GC phases. 2) Lightweight Locking via Stack Displaced Mark Word: A thread copies the target object's Mark Word to its local stack frame (Displaced Mark Word) and performs a CAS to store the stack address in the object header. If successful, acquisition completes with zero kernel transitions. 3) Adaptive Spinning: Before invoking heavy `pthread_mutex_lock` (which suspends the thread), the JVM spins the core for several clock cycles. If the holding thread releases during the spin, execution proceeds seamlessly without context-switch latency.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Kilit Çekişmesi ve Adaptif Spinning Simülasyonu
public class LockInflationDemo {
    private final Object monitor = new Object();
    private int counter = 0;

    // Çekişmesiz: Hafif Kilit (Lightweight CAS) olarak çalışır
    public void uncontendedOperation() {
        synchronized (monitor) {
            counter++;
        }
    }

    // Yüksek Çekişme: Ağır Kilit (Heavyweight Monitor Inflation) tetikler
    public void contendedOperation() {
        synchronized (monitor) {
            try {
                Thread.sleep(1); // Bilerek kilit içinde beklenir, diğerleri şişer!
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            counter++;
        }
    }

    void main() {
        var demo = new LockInflationDemo();
        demo.uncontendedOperation();
        IO.println("Hafif kilit başarıyla çalıştı: " + demo.counter);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Klasiği: 'Thread.sleep() kilit içindeyken kilidi bırakır mı?' Yanıt: 'KESİNLİKLE BIRAKMAZ! sleep() kilidi tutmaya devam ederken diğer tüm thread'leri kapıda bloke eder. Kilidi geçici olarak bırakan metot Object.wait()'tir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Crucial Interview Question: 'Does Thread.sleep() release held synchronized monitors?' Answer: 'ABSOLUTELY NOT! `Thread.sleep()` sleeps while retaining ownership of every single held lock, starving other threads. Only `Object.wait()` atomically releases the monitor.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- HotSpot kilitleri çekişmeye göre Lightweight'ten Heavyweight'e şişer (Inflate).
- Adaptif dönme (Adaptive Spinning), thread uyutma (park) maliyetini önler.
- Thread.sleep() kilidi ASLA bırakmaz; Object.wait() ise kilidi bırakır.

🎯 Key Takeaways for the Interview

- HotSpot locks dynamically inflate from Lightweight CAS to Heavyweight OS monitors.
- Adaptive spinning attempts to acquire monitors during spin loops before sleeping.
- Thread.sleep() retains locks; Object.wait() atomically surrenders them.

15

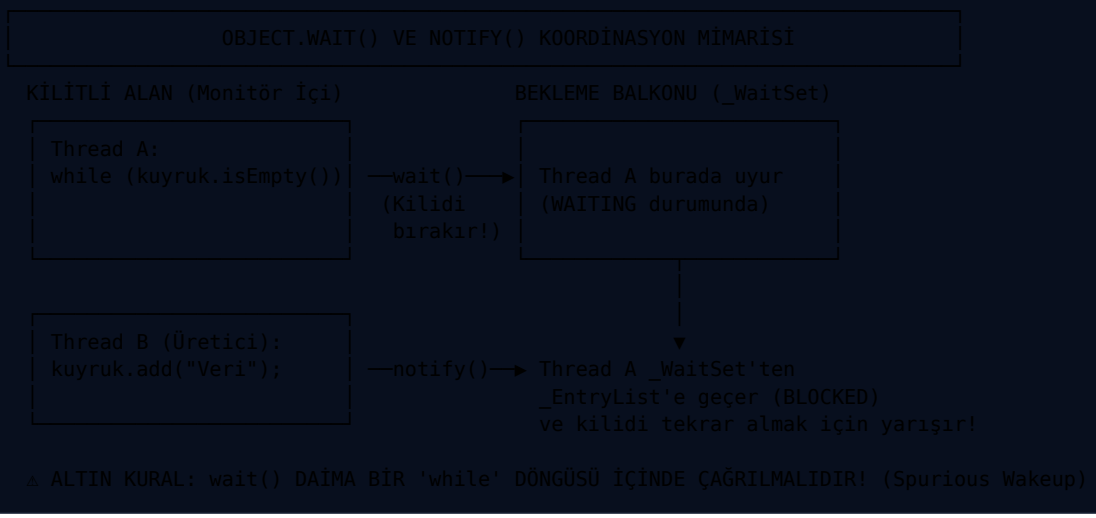
BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

Thread'ler Arası İletişim: wait(), notify(), notifyAll() & Kaçan Sinyal (Missed Signal)

Inter-Thread Signaling: wait(), notify(), notifyAll() & The Missed Signal Hazard

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranda müşteri (Tüketici) masaya oturuyor. Garsona 'Bana yemek getir' diyor. Mutfakta yemek yoksa müşteri masayı meşgul etmez (wait() çağırıp kilidi bırakır) ve kenardaki bekleme salonuna geçer. Aşçı (Üretici) yemeği pişirdiği anda 'Yemek hazır!' diye zili çalar (notify()). Müşteri zil sesini duyunca hemen gidip masaya oturmaz; önce tabağa bakar: 'Gerçekten yemek geldi mi, yoksa garson yanlışlıkla mı zile bastı?' (while döngüsü). Eğer yemek varsa yer; yoksa tekrar beklemeye geçer!

🇬🇧 ENGLISH (ELI5)

Imagine a patron at a diner. If the soup pot is empty, the customer cannot sit occupying the single service stool. They step aside to the lounge via `wait()`, atomically relinquishing the stool. When the chef brews fresh soup, the chef rings the service bell via `notify()`. The patron awakes, but does NOT blindly eat. They inspect the pot inside a `while` loop: 'Is there soup, or did another patron grab it first, or did lightning ring the bell (Spurious Wakeup)?' If soup exists, they eat; otherwise, they wait again.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

wait/notify Koordinasyonunda 3 Hayati Kural: 1) Neden synchronized içinde olmalıdır?: `wait()` veya `notify()` kilitsiz çağrılırsa JVM anında `IllegalMonitorStateException` fırlatır; çünkü paylaşılan durumu kontrol etmek ve sinyal göndermek atomik bir bütün olmak zorundadır. 2) Spurious Wakeup (Sahte Uyanma): İşletim sistemi sinyalleri veya kernel kesintileri yüzünden bir thread hiçbir `notify()` çağrılmasa dahi `wait()`ten uyanabilir. Bu yüzden `if (condition) wait();` yazmak ölümcül bir hatadır; DAİMA `while (condition) wait();` yazılmalıdır! 3) notify() vs notifyAll(): `notify()` rastgele tek bir thread uyandırır; uyandırılan thread işi yapamazsa tüm sistem sonsuza dek kilitlenebilir (Deadlock / Missed Signal). Bu yüzden özel bir optimizasyon gerekmiyorsa her zaman `notifyAll()` kullanılmalıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Mechanics of Condition Signaling: 1) Prerequisite Monitor Ownership: Invoking `wait()` or `notify()` outside of a synchronized block targeting that exact object immediately throws `IllegalMonitorStateException`. 2) Spurious Wakeups: Under POSIX threading implementations, waiting threads can awaken absent any explicit notification due to OS-level signal handling. Guarding `wait()` with an `if` statement causes race conditions where unverified invariants execute; `while (!condition) { wait(); }` is mathematically required. 3) The `notify()` vs `notifyAll()` Dilemma: `notify()` signals an arbitrary thread from `_WaitSet`. If that thread is unable to satisfy state requirements, the notification is lost, hanging the entire pool. `notifyAll()` broadcasts to all waiting threads, shifting them to `_EntryList`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Klasik Producer-Consumer (Üretici-Tüketici) Kuyruğu
import java.util.LinkedList;
import java.util.Queue;

public class BoundedBlockingQueue<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;

    public BoundedBlockingQueue(int capacity) {
        this.capacity = capacity;
    }

    public synchronized void put(T item) throws InterruptedException {
        // ⚠ DAİMA WHILE DÖNGÜSÜ! Spurious Wakeup koruması
        while (queue.size() == capacity) {
            wait(); // Kuyruk dolu, yer açılana kadar kilidi bırakıp bekle
        }
        queue.add(item);
        notifyAll(); // Tüketicilere haber ver: Veri geldi!
    }

    public synchronized T take() throws InterruptedException {
        while (queue.isEmpty()) {
            wait(); // Kuyruk boş, veri gelene kadar bekle
        }
        T item = queue.remove();
        notifyAll(); // Üreticilere haber ver: Yer açıldı!
        return item;
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'Neden if yerine while kullanıyoruz?' diye sorarsa: '1) Sahte uyanmalar (Spurious Wakeup) yüzünden thread sinyal olmadan uyanabilir. 2) notifyAll çağrıldığında uyandırılan diğer bir thread araya girip kuyruktaki elemanı kapmış olabilir. Bu yüzden uyanan her thread koşulu tekrar doğrulamak zorundadır!' denmelidir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Mandatory Interview Answer: 'Why must wait() always sit in a while loop rather than an if block?' Answer: '1) Spurious wakeups allow threads to awaken without signals. 2) When notifyAll awakens multiple threads, another thread may race ahead and consume the state first. The awakening thread must re-evaluate the invariant before proceeding.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- wait(), notify() ve notifyAll() sadece synchronized blok içinde çağrılabilir.
- Sahte uyanmalara (Spurious Wakeup) karşı wait() daima while döngüsü içinde olmalıdır.
- Kaçan sinyal (Missed Signal) hatalarını önlemek için varsayılan olarak notifyAll seçilir.

🎯 Key Takeaways for the Interview

- wait() and notify() require active monitor ownership; otherwise IllegalMonitorStateException.
- Always guard wait() with while(!condition) to defend against spurious wakeups.
- Favor notifyAll() over notify() to prevent lost signal deadlocks.

16

BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

ReentrantLock: Açık Kilitler, Adil (Fair) vs Haksız (Non-Fair) & Kesilebilirlik

ReentrantLock: Explicit Locking, Fair vs Non-Fair & Lock Interruptibility

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
SYNCHRONIZED vs REENTRANTLOCK MİMARİ KARŞILAŞTIRMASI

ÖZELLİK                 SYNCHRONIZED              REENTRANTLOCK
-------------------------------------------------------------------
Kilit Bırakma           Otomatik (Compiler)       Manuel (finally { unlock(); })
Zaman Aşımı (Timeout)   YOK (Sonsuza dek bekler)  VAR: tryLock(5, SECONDS)
Kesilebilirlik          YOK (Kesilemez)           VAR: lockInterruptibly()
Adillik (Fairness)      Sadece Haksız (Barging)   Fair / Non-Fair seçilebilir
Çoklu Kuyruk            Tek bir WaitSet           Birden fazla Condition nesnesi

NON-FAIR LOCK (Haksız Kilit - Yüksek Throughput):
Yeni gelen thread kuyrukta bekleyenleri beklemeden doğrudan kilide saldırır (Barging)!
Context switch maliyetinden kaçar, %400 daha hızlıdır!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Geleneksel anahtarlar (synchronized) basittir; kapıyı açar girersiniz, çıkınca kapı arkadan otomatik kilitlenir. Ama diyelim ki kapının önünde 3 saat beklemek istemiyorsunuz: 'Eğer 10 saniye içinde açılmazsa ben gidiyorum' demek istiyorsunuz (`tryLock(timeout)`). Ya da yangın çıktığında bekleyen adamı kolundan çekip dışarı çıkarmak istiyorsunuz (`lockInterruptibly()`). İşte ReentrantLock, cebinizde zamanlayıcısı, acil durum butonu ve sıra numarası olan son teknoloji akıllı kilit sistemidir!

🇬🇧 ENGLISH (ELI5)

An intrinsic lock (`synchronized`) is an old-fashioned deadbolt: you wait until it opens, with no escape. ReentrantLock is an enterprise smart lock: 1) You can say: 'If the door doesn't unlock in 5 seconds, I'm leaving' (`tryLock(timeout)`). 2) In a fire alarm, a thread waiting for the lock can be canceled immediately (`lockInterruptibly()`). 3) You can enforce a strict first-come-first-served line (Fairness) or allow energetic newcomers to barge in immediately for 4x higher throughput (Non-Fair).

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ReentrantLock ve AQS Derinliği: 1) AbstractQueuedSynchronizer (AQS): ReentrantLock, temelinde çift yönlü bir FIFO bekleme kuyruğu (CLH lock varyantı) ve bir `volatile int state` tutar. 2) Fair vs Non-Fair Kilit: Non-fair kilit (varsayılan) 'Barging' uygular. Yeni gelen thread, kuyrukta bekleyenler olmasına rağmen önce bir CAS ile kilidi kapmaya çalışır. Eğer o an kilit boştaysa, kuyruk uyandırma (unpark) ve context-switch gecikmesini atlayarak kilidi alır; throughput dramatik yükselir. Fair lock ise daima kuyruk başındaki thread'e verir, adaleti sağlar ancak gecikmeyi katlar. 3) finally Bloğu Zorunluluğu: `lock.unlock()` daima `try-finally` bloğunun finally kısmında olmalıdır; aksi halde bir exception fırladığında kilit sonsuza dek kilitli kalır ve sistem felç olur!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architectural Internals of ReentrantLock: 1) AQS Foundation: ReentrantLock wraps `Sync extends AbstractQueuedSynchronizer`. State 0 represents unlocked; state >= 1 denotes reentrant holds by the owning thread. 2) The Non-Fair Barging Advantage: `NonfairSync` attempts an immediate CAS on `state` before enqueuing. If an unparking thread has not yet acquired the CPU core, a newly arriving active thread sneaks in, eliminating two context-switch penalties. This yields massive throughput gains at the expense of potential thread starvation. 3) Non-Negotiable Idiom: Because the compiler cannot enforce automatic release, `lock.lock()` must immediately precede a `try` block, with `lock.unlock()` residing strictly inside `finally`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: ReentrantLock ile Zaman Aşımlı (tryLock) ve Kesilebilir Kilit
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.ReentrantLock;

public class AdvancedLockingDemo {
    private final ReentrantLock lock = new ReentrantLock(false); // Non-fair (Hızlı)

    public void processPayment() {
        lock.lock();
        try {
            IO.println("Ödeme işlemi kilitlendi: " + Thread.currentThread().getName());
        } finally {
            lock.unlock(); // ⚠ KESİNLİKLE finally içinde olmalıdır!
        }
    }

    public boolean tryProcessWithTimeout() throws InterruptedException {
        // 2 saniye içinde kilit alamazsa vazgeç (Deadlock önleyici!)
        if (lock.tryLock(2, TimeUnit.SECONDS)) {
            try {
                IO.println("Zaman aşımlı kilit alındı!");
                return true;
            } finally {
                lock.unlock();
            }
        } else {
            IO.println("Kilit meşguldü, işlem timeout'a uğradı!");
            return false;
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Ölümcül Kodlama Hatası: `lock.lock()` satırını `try` bloğunun İÇİNE yazmayın! Eğer `lock()` anında bir hata fırlatırsa, `finally` bloğu çalışır ve sahip olunmayan bir kilit için `unlock()` çağrılır (`IllegalMonitorStateException`). Kural: `lock.lock(); try { ... } finally { lock.unlock(); }`!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Severe Anti-Pattern: Never place `lock.lock()` INSIDE the `try` block! If `lock()` throws before acquisition, `finally` executes `unlock()`, triggering an `IllegalMonitorStateException` that masks the root cause. Idiom: Invoke `lock.lock()` strictly on the line preceding `try`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ReentrantLock; tryLock(), timeout ve interruptibility esnekliği sunar.
- Non-fair kilitler context switch maliyetini baypas ettiği için çok daha yüksek throughput üretir.
- unlock() çağrısı DAİMA finally bloğu içerisinde yer almalıdır.

🎯 Key Takeaways for the Interview

- ReentrantLock provides tryLock(), timeouts, and interruption capabilities.
- Non-fair locks bypass context switches via barging, delivering superior throughput.
- unlock() must strictly execute inside a finally block to prevent resource leaks.

17

BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

Condition Değişkenleri: Çoklu Bekleme Kuyrukları (notFull & notEmpty)

Condition Variables: Multi-Queue Coordination (notFull & notEmpty)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              TEK BİR REENTRANTLOCK İÇİNDE ÇİFT BEKLEME KUYRUĞU              │
└──────────────────────────────────────────────────────────┘

ReentrantLock lock = new ReentrantLock();
Condition notFull  = lock.newCondition(); // Üreticilerin bekleme kuyruğu
Condition notEmpty = lock.newCondition(); // Tüketicilerin bekleme kuyruğu

[ ÜRETİCİ (PUT) ]:
Kuyruk doluysa ──► notFull.await() ──► Sadece notFull kuyruğunda bekler!
Eleman ekleyince ──► notEmpty.signal() ──► YALNIZCA Tüketicileri uyandırır!

[ TÜKETİCİ (TAKE) ]:
Kuyruk boşsa ──► notEmpty.await() ──► Sadece notEmpty kuyruğunda bekler!
Eleman çekince ──► notFull.signal() ──► YALNIZCA Üreticileri uyandırır!

AVANTAJ: synchronized gibi herkesi tek bir torbaya atıp uyandırmaz (Sıfır İtişme)!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski sistemde (synchronized wait/notify) lokantada hem yemek bekleyen aç müşteriler hem de boş tabak bekleyen bulaşıkçılar aynı odada bekliyordu. Aşçı 'Yemek hazır' dediğinde herkesi aynı anda uyandırıyordu (notifyAll). Bulaşıkçılar uyanıp 'Bize yemek lazım değil ki' diyip tekrar uyuyordu (Boşa enerji). Condition ise lokantaya iki ayrı kapı koymaktır: 1. Kapı: 'Yemek Bekleyenler' (notEmpty). 2. Kapı: 'Temiz Tabak Bekleyenler' (notFull). Aşçı yemek yapınca sadece müşterilerin kapısını çalar; bulaşıkçıları boş yere uyandırmaz!

🇬🇧 ENGLISH (ELI5)

In intrinsic `wait/notify`, producers and consumers share a single noisy waiting room. When a producer delivers an item and calls `notifyAll()`, both idle producers and consumers wake up, frantically thrashed from sleep only to realize the notification wasn't for them. `Condition` splits the room into targeted queues: `notFull` for producers and `notEmpty` for consumers. Producers wake only consumers; consumers wake only producers. Zero wasted CPU cycles!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Condition Mimarisi ve ArrayBlockingQueue: 1) Çoklu Bekleme Seti: Tek bir kilit üzerinden `lock.newCondition()` ile onlarca farklı koşul kuyruğu oluşturulabilir. 2) await() ve signal(): `await()` kilidi bırakıp thread'i o Condition'a özel AQS bekleme kuyruğuna sokar. `signal()` ise sadece o kuyruğun başındaki tek bir thread'i kilit için yarışmaya gönderir. 3) Neden signal() güvenlidir?: Çünkü `notEmpty.signal()` çağrıldığında uyandırılan kişinin KESİNLİKLE bir tüketici olduğu garanti altındadır! `notify()` gibi yanlışlıkla başka bir üreticiyi uyandırıp sistemi kilitleme tehlikesi yoktur. JDK'nın `ArrayBlockingQueue` sınıfı tamamen bu desenle yazılmıştır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Condition Queues and ArrayBlockingQueue Topology: 1) Dedicated AQS Wait Nodes: Each `ConditionObject` instance maintains its own first/last FIFO node pointers. When a thread executes `await()`, AQS saves the hold count, fully releases the lock, and enqueues the thread in the condition queue. 2) Targeted Signaling: Calling `signal()` unlinks the first node from the condition queue and transfers it to the AQS sync queue (`_EntryList` equivalent). 3) Safety of `signal()` over `signalAll()`: Because distinct conditions cleanly isolate producers from consumers, calling `notEmpty.signal()` is 100% immune to the missed-signal hazards that plague intrinsic `notify()`. This is the exact implementation powering `java.util.concurrent.ArrayBlockingQueue`.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: Condition Değişkenleri ile Yüksek Başarımlı Bounded Queue
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public class ConditionQueue<T> {
    private final Object[] items;
    private int count, putIndex, takeIndex;

    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull  = lock.newCondition(); // Üretici kuyruğu
    private final Condition notEmpty = lock.newCondition(); // Tüketici kuyruğu

    public ConditionQueue(int capacity) {
        this.items = new Object[capacity];
    }

    public void put(T x) throws InterruptedException {
        lock.lock();
        try {
            while (count == items.length) {
                notFull.await(); // Yer yoksa üretici bekler
            }
            items[putIndex] = x;
            if (++putIndex == items.length) putIndex = 0;
            count++;
            notEmpty.signal(); // Sadece tüketicilere sinyal ver!
        } finally {
            lock.unlock();
        }
    }

    @SuppressWarnings("unchecked")
    public T take() throws InterruptedException {
        lock.lock();
        try {
            while (count == 0) {
                notEmpty.await(); // Veri yoksa tüketici bekler
            }
            T x = (T) items[takeIndex];
            if (++takeIndex == items.length) takeIndex = 0;
            count--;
            notFull.signal(); // Sadece üreticilere sinyal ver!
            return x;
        } finally {
            lock.unlock();
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'Condition.await() çağrılmadan önce lock.lock() çağrılmazsa ne olur?' Yanıt: 'Tıpkı synchronized olmadan wait() çağrıldığında olduğu gibi IllegalMonitorStateException fırlatılır! Condition'lar ait oldukları ReentrantLock'a göbekten bağlıdır.'

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Requirement: Invoking `condition.await()` or `condition.signal()` without holding the associated `ReentrantLock` immediately throws `IllegalMonitorStateException`. The calling thread must hold the lock that owns the condition.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Condition, tek bir kilit altında birden fazla bağımsız bekleme kuyruğu oluşturur.

• notFull ve notEmpty ayrımı sayesinde signal() güvenle kullanılabilir.

• JDK'nın ArrayBlockingQueue sınıfı Condition mimarisi üzerine inşa edilmiştir.

🎯 Key Takeaways for the Interview

• Condition variables decouple wait-sets into distinct semantic queues.

• Condition queues make signal() completely safe by targeting specific thread roles.

• ArrayBlockingQueue is the canonical enterprise implementation of Condition queues.

18

BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

ReentrantReadWriteLock: Okuma Yoğun Sistemler & Kilit Düşürme (Lock Downgrading)

ReentrantReadWriteLock: Read-Intensive Scaling & Lock Downgrading Mechanics

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              REENTRANTREADWRITELOCK: PAYLAŞIMLI vs ÖZEL KİLİT │
└──────────────────────────────────────────────────────────┘

OKUMA KİLİDİ (ReadLock - Shared):

┌──────────┐  ┌──────────┐  ┌──────────┐
│ Thread 1 │  │ Thread 2 │  │ Thread 3 │ ──► AYNI ANDA BİNLERCE THREAD OKUYABİLİR!
└──────────┘  └──────────┘  └──────────┘

YAZMA KİLİDİ (WriteLock - Exclusive / Mutex):

┌──────────┐
│ Thread W │ ──► SADECE TEK BİR THREAD YAZABİLİR! (Tüm okuyucular bloke edilir)
└──────────┘

LOCK DOWNGRADING (Kilit Düşürme):
WriteLock AL ──► Veriyi Güncelle ──► ReadLock AL ──► WriteLock BIRAK ──► Okumaya Devam Et!
(Veri tutarlılığını kaybetmeden güvenli kilit seviyesi düşürme)

⚠ DİKKAT: Lock Upgrading (Okumadan Yazmaya Geçiş) DESTEKLENMEZ! (Deadlock Yaratır!)
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphanedeki büyük ansiklopedi düşünün. Aynı anda 100 öğrenci gelip ansiklopediyi sessizce okuyabilir (Okuma Kilidi / ReadLock); kimse kimseyi engellemez. Ama ansiklopedideki hatalı bir sayfayı yırtıp yenisini yapıştıracak olan matbaacı (Yazma Kilidi / WriteLock) geldiğinde, kütüphanedeki bütün öğrencileri dışarı çıkartır ve kapıyı kilitler! İşini bitirince kapıyı açar ve öğrenciler tekrar içeri dolar. Okuma çok, yazma azsa sistem roket gibi hızlanır!

🇬🇧 ENGLISH (ELI5)

Imagine a public stock ticker display. Thousands of traders can stare at the display concurrently without interfering with one another (Shared Read Lock). However, when the exchange operator updates the asset prices (Exclusive Write Lock), everyone must pause until the prices settle. If 99% of requests are reads and 1% are writes, a ReadWriteLock boosts concurrency a thousandfold compared to a blunt mutex!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ReadWriteLock Mimarisi ve Kilit Düşürme Kuralları: 1) Durum Yönetimi (State Split): AQS'in 32-bitlik tek `state` tamsayısı ikiye bölünür: Üst 16 bit okuma kilidi sayısını (Shared count), alt 16 bit ise yazma kilidi sayısını (Exclusive count) tutar. 2) Lock Downgrading (Kilit Düşürme): Yazma kilidini tutarken önce okuma kilidi alınır, ardından yazma kilidi bırakılır. Bu sayede yazma ile okuma arasına başka bir yazıcının girmesi engellenerek veri tazeliği garanti edilir. 3) Lock Upgrading Neden Yoktur?: Eğer iki okuyucu thread aynı anda yazma kilidine yükselmek (upgrade) isterse, ikisi de birbirinin okuma kilidini bırakmasını bekler ve anında kilitlenir (Deadlock). Bu yüzden Java'da kilit yükseltme yasaktır!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ReadWriteLock Internals & Downgrade Protocols: 1) 32-Bit State Partitioning: AQS encodes dual locks within a single atomic integer: the high 16 bits record shared read acquisitions; the low 16 bits record exclusive write hold counts. 2) Lock Downgrade Sequence: While holding the write lock, acquire the read lock, then release the write lock. This preserves atomic continuity; no competing writer can slip in between the write release and read acquisition. 3) The Ban on Lock Upgrading: Upgrading from read lock to write lock is strictly disallowed. If Thread A and Thread B both hold read locks and attempt to upgrade, both freeze waiting for the other to release, creating an immediate deadlock.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Lock Downgrading (Kilit Düşürme) Deseni
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class CacheWithLockDowngrade {
    private final ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock();
    private String cachedData = "Initial Data";
    private boolean cacheValid = false;

    public void processData() {
        rwLock.readLock().lock();
        if (!cacheValid) {
            // Önce okuma kilidini bırakmak zorundayız!
            rwLock.readLock().unlock();

            // Yazma kilidini al
            rwLock.writeLock().lock();
            try {
                if (!cacheValid) { // Re-check
                    cachedData = "Fresh Data from DB";
                    cacheValid = true;
                }
                // ⚠ KİLİT DÜŞÜRME (LOCK DOWNGRADE) BURADA BAŞLAR:
                rwLock.readLock().lock(); // 1. Okuma kilidini AL
            } finally {
                rwLock.writeLock().unlock(); // 2. Yazma kilidini BIRAK (Kilit düşürüldü!)
            }
        }

        try {
            // Veriyi güvenle oku
            IO.println("İşlenen Veri: " + cachedData);
        } finally {
            rwLock.readLock().unlock(); // 3. Son okuma kilidini BIRAK
        }
    }

    void main() {
        new CacheWithLockDowngrade().processData();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'ReentrantReadWriteLock kullanırken okuma kilidi içindeyken yazma kilidi alabilir miyiz (Lock Upgrading)?' diye sorar. Yanıt: 'HAYIR! Kilit yükseltme (Lock Upgrading) Deadlock riski nedeniyle Java'da desteklenmez; thread sonsuza dek kilitlenir. Ancak Kilit Düşürme (Lock Downgrading) tamamen desteklenir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Classic MAANG Trap: 'Can a thread holding a read lock directly upgrade to a write lock?' Senior Answer: 'NO! Lock upgrading is strictly prohibited in Java to prevent deadlocks. Attempting to acquire a write lock while holding a read lock deadlocks the thread forever. However, Lock Downgrading (write to read) is fully supported.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ReadWriteLock, okuma yoğun (%90+ okuma) sistemlerde devasa verim artışı sağlar.
- Kilit Düşürme (Downgrade: Write -> Read) güvenle desteklenir.
- Kilit Yükseltme (Upgrade: Read -> Write) deadlock nedeniyle yasaktır.

🎯 Key Takeaways for the Interview

- ReadWriteLock shines in read-heavy workloads (90%+ reads).
- Lock Downgrading (WriteLock -> ReadLock -> unlock WriteLock) is fully supported.
- Lock Upgrading is strictly prohibited due to inherent cyclic deadlocks.